CS 1910

Computer Science I

Lecture 6

Decisions

QUESTIONS

- Questions from last day?

CODE EXAMPLES

- https://colab.research.google.com/github/cdspower-upei/2026W-CS1910-Students/blob/main/Lectures/L06/L06-Examples.ipynb

BOOLEAN

- Recall that Boolean values are either True or False
- Boolean expressions evaluate to a Boolean value

Logical Operators	Relational (comparison) Operators
`and`	`==`
`or`	`>=`
`not`	`<=`
	`!=`
	`>`
	`<`

QUICK HITTERS

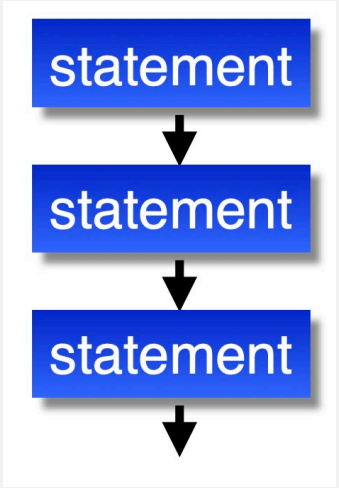
- What is the result of the expression

```
42 != 42
42 <= 42
“A” < “A” or “BB” < “BBB”
1 - 1 > 0 and -2 > 0
len(“ABC”) > 0 and not len(“ABC”) < 2
```

*Logical operators have a lower precedence than Relational operators

FLOW OF CONTROL

- Typically the flow of our programs has been a sequence of statements executed in order
- There are statements that alter this order:



Conditional Statements

Repetition Statements

CONDITIONAL STATEMENTS

- Uses a Boolean expression to determine which statements are executed next (or not executed at all)
- Programmers use these to program decision logic
- Main types of Conditional Statements:

if Statement

if-else Statement

If-elif-else Statement

IF STATEMENT

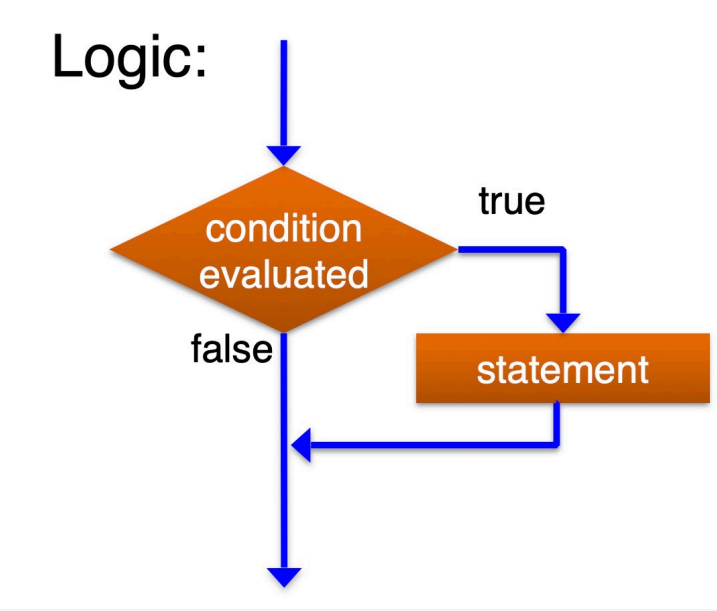
Code that runs when the condition is True

Must be indented in a code block

Boolean expression or value

```
if condition:
    # if block (indented)

# code after else block
```

End of indentation indicates end of the if block

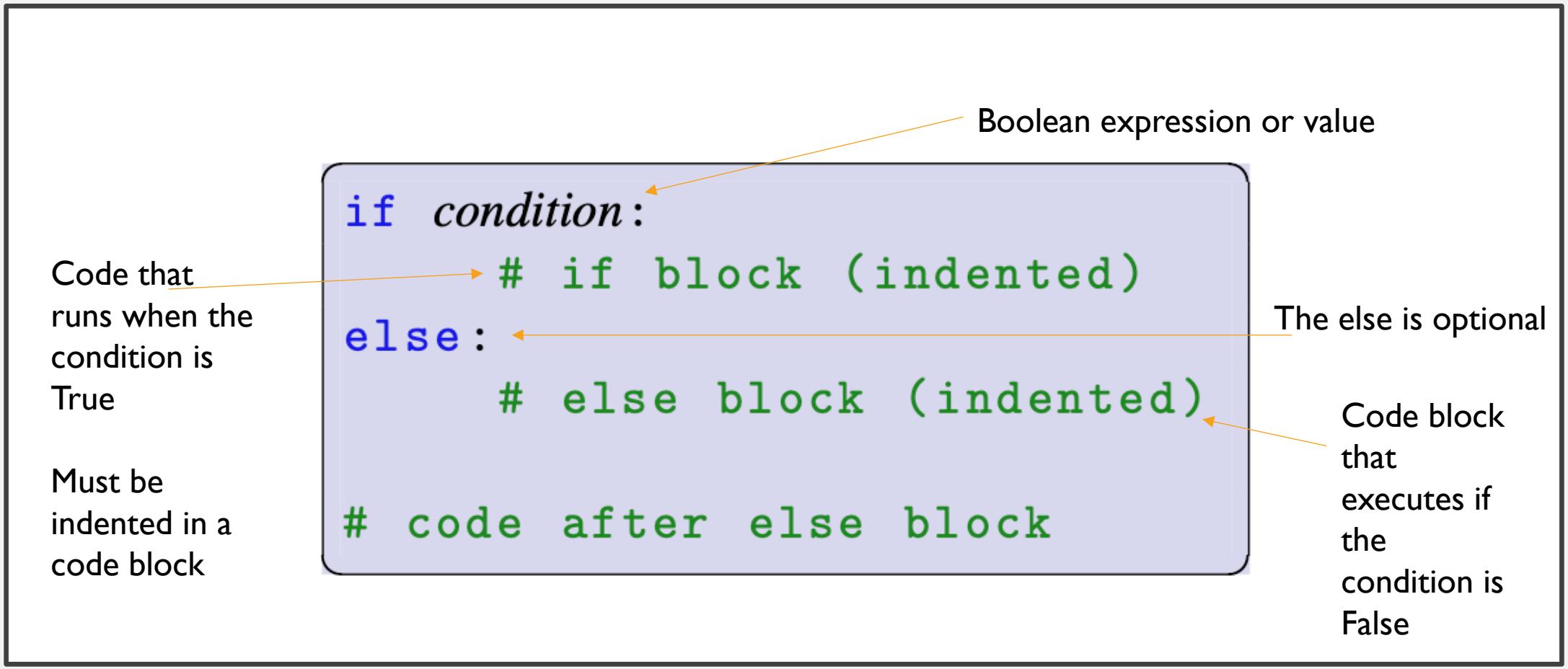


EXAMPLE

```
# if statement

bonus = 6
grade = 72

if bonus > 5:
    grade += bonus

print(f"grade: {grade}")
```

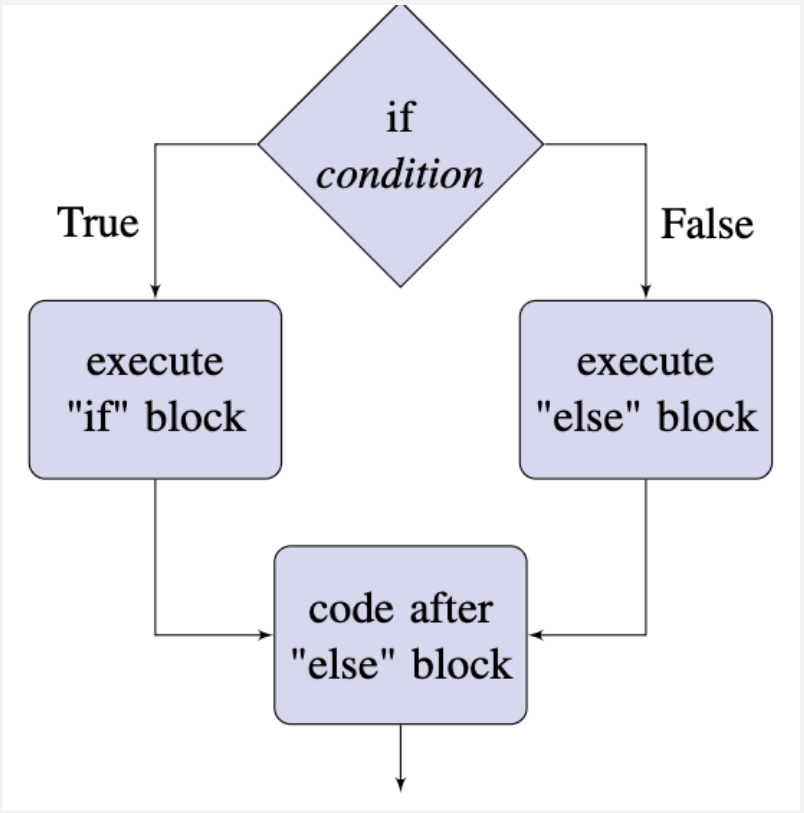
```
grade: 78
```

IF-ELSE STATEMENT



IF-ELSE STATEMENT

```
if condition:
        # if block (indented)
else:
        # else block (indented)

# code after else block
```



SELF-CHECK

- We want to detect that someone hasn't rolled double 6's:

```
# not double 6's
die_roll1 = 5
die_roll2 = 6

if not die_roll1 ==6 and not die_roll2 == 6:
    print("not 12 the hard way")
else:
    print("ack double 6")
```

What's the output?

SELF-CHECK

- We want to detect that someone hasn't rolled double 6's:

```
# not double 6's
die_roll1 = 5
die_roll2 = 6

if not die_roll1 ==6 and not die_roll2 == 6:
    print("not 12 the hard way")
else:
    print("ack double 6")
```

ack double 6 ← Why?

Can you fix the logic?

```python
#  if - else statement

shoe_size = 7
price = 50

if shoe_size < 5:
    #childrens clothing tax
    tax = 1.09
else:
    tax = 1.15

price *= tax

print(f"tax: {tax:.2f}, price: {price:.2f}")
```

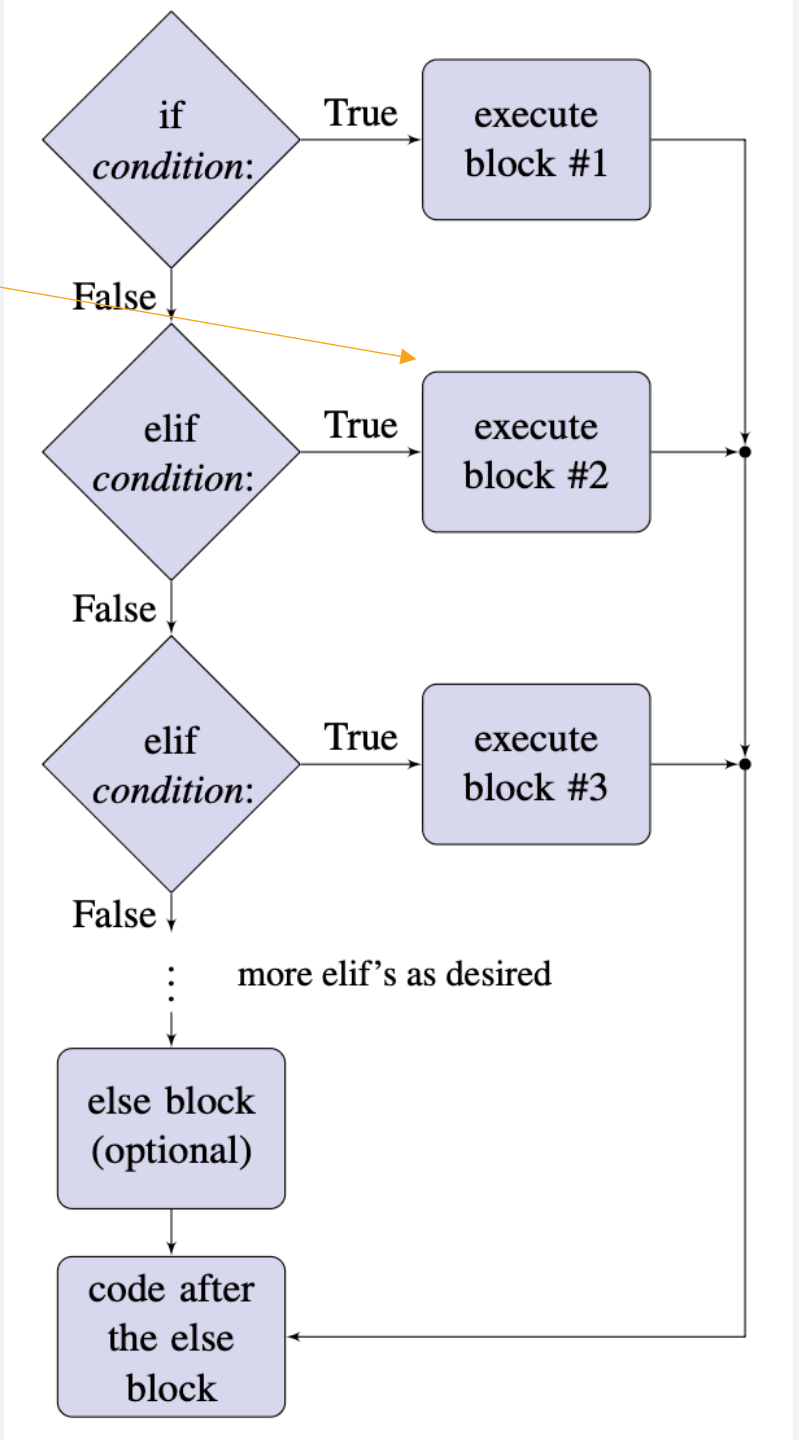
```
tax: 1.15, price: 57.50
```

EXAMPLE

IF-ELIF-ELSE STATEMENT

Only one of these code blocks is executed

```
if condition:
        # block 1 (indented)
elif condition:
        # block 2 (indented)
elif condition:
        # block 3 (indented)

# ... more elif's as desired

else:   # (optional)
        # else block

# code after the else block
```



EXAMPLE

```
# if-elif-else statement

grade = 78
letter_grade = ""

if grade > 90:
    letter_grade = "A"
elif grade > 75:
    letter_grade = "B"
elif grade > 60:
    letter_grade = "C"
elif grade > 50:
    letter_grade = "D"
else:
    letter_grade = "F"

print(f"grade: {grade} ({letter_grade})")
```

```
grade: 78 (B)
```

CLASS PARTICIPATION

Check-in about last class on
Input/Output and Built-in Functions

2026W CS1910 05 In-Class Participation
Input/Output and Built-ins – Fill out form

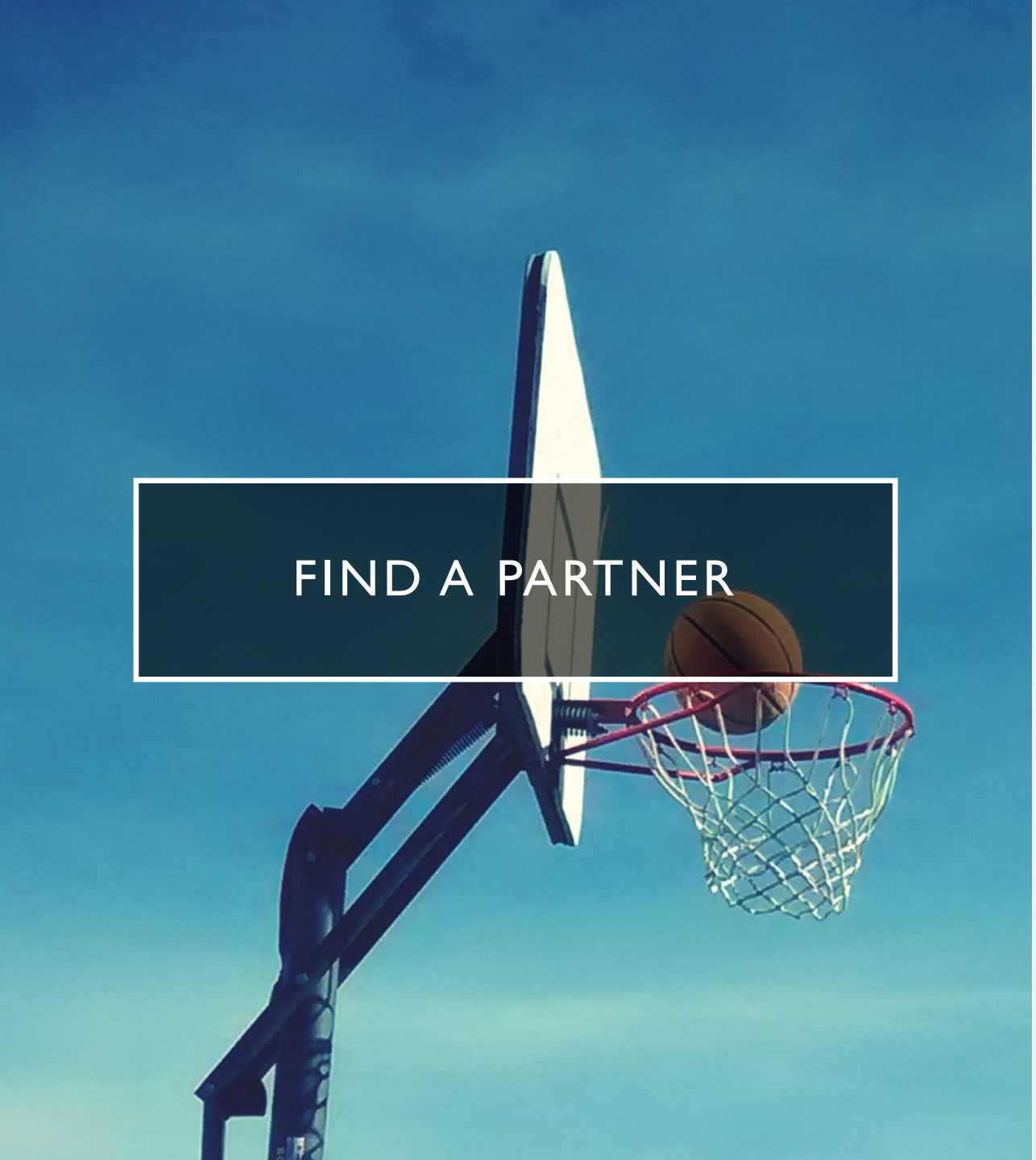
NESTED CONDITIONAL STATEMENTS

Code block at new indentation level

- Any of the code blocks associated with if, elif or else can also contain a conditional statement (an if …)
- We call these nested conditionals or nested if statements
- The nested if (inside an if code block) gets its own indented code block

```
if temperature > 20:
    if not windy:
        print ("Perfect weather, lets go to the beach!")
    else:
        print("The temperature is nice, but it is too windy.")
else:
    print("It is too cold, I'll stay home.")
```

- Complete the bank balance example
- A customer wants to withdraw money from their account
- If they try to withdraw more money than they have you should print an error saying insufficient funds, unless the customer has overdraft protection in which case they are allowed to carry a negative balance.
- After the transaction print the balance (regardless as to whether it happened or not



FIND A PARTNER

- Complete the Basketball example
- In the WNBA
- if you are fouled while taking a shot and your shot does not go in, you get the equivalent number of free throws as your shot was worth (2 for a 2 pointer and 3 for a 3 pointer).
- If your shot does go in (and you were fouled) you get the appropriate amount of points plus 1 additional free throw.
- If you miss your shot and no foul occurs you get no points
- If you make your shot and no foul occurs you get the appropriate number of points

FIND A PARTNER

- Brain storm another real-world example where we might have complex logic or nested if statements to model it in software

COMPARISONS PAUSE

A few notes about comparisons

FLOATING POINT VALUES

- Usually we do not use == to compare floating point values
- Floating point numbers can be challenging to fully represent in a discrete finite binary system
- Just like 1/3 is challenging to represent in decimal, various fractions are challenging in binary (such as 0.1) as they produce long or repeating sequences
- This leads to rounding errors and makes it unreliable to compare directly two floating point numbers
- Example:

```
#how many pennies (former name for one cent coins) is $72.99?
money = 72.99
cents = money * 100
print(cents)
```

```
7298.999999999999
```

WHAT IS THE OUTPUT?

```python
# Consider the following
# what is the output

invoice = 18.58
first_payment = 18.10
second_payment = 0.48

remainder = invoice - (first_payment + second_payment)

print(f"invoice: {invoice}")
print(f"total payments: {first_payment + second_payment}")

if remainder == 0:
    print("invoice paid")
else:
    print("more money to pay")
```

WHAT IS THE OUTPUT?

Whoops! That's not right!

```python
# Consider the following
# what is the output

invoice = 18.58
first_payment = 18.10
second_payment = 0.48

remainder = invoice - (first_payment + second_payment)

print(f"invoice: {invoice}")
print(f"total payments: {first_payment + second_payment}")

if remainder == 0:
    print("invoice paid")
else:
    print("more money to pay")
```

```
invoice: 18.58
total payments: 18.580000000000002
more money to pay
```

COMPARING FLOATS

- 2 Solutions to avoid direct comparisons
  - Use a threshold
  - Use a library

THRESHOLD

- Rather than writing `float_variable_1 == float_variable_2`
- Subtract and compare against a small threshold of acceptance
  - `float_variable_1 - float_variable_2 < threshold`
- GOTCHA … Use ABS to avoid issues with negatives

```
threshold = 0.0000001
abs(float_variable_1 - float_variable_2) < threshold
```

MATH.ISCLOSE

- The math library has a method isclose that determines if 2 floating point numbers are really close
  - Has a default tolerance of 9 decimal places but can be configured

```
math.isclose(float_variable_1, float_variable_2)
```

COMPARING FLOATS

Fix the code to avoid directly comparing floating points

```python
# Consider the following
# what is the output

invoice = 18.58
first_payment = 18.10
second_payment = 0.48

remainder = invoice - (first_payment + second_payment)

print(f"invoice: {invoice}")
print(f"total payments: {first_payment + second_payment}")

if remainder == 0:
    print("invoice paid")
else:
    print("more money to pay")
```

COMPARING FLOATS

Fix the code to avoid directly comparing floating points

```python
# Fix the example to use a threshold or math.isclose

invoice = 18.58
first_payment = 18.10
second_payment = 0.48

remainder = invoice - (first_payment + second_payment)

print(f"invoice: {invoice}")
print(f"total payments: {first_payment + second_payment}")

threshold = 0.00000001
if abs(remainder - 0) < threshold:
    print("invoice paid")
else:
    print("more money to pay")
```

```
invoice: 18.58
total payments: 18.580000000000002
invoice paid
```

COMPARING STRINGS

- Characters are represented as numbers (binary numbers)
- For example “A” is character 65,
- Strings are compared by their binary code (ordinal number)

```
cook@pop-os:~$ ascii -d
   0 NUL     16 DLE     32          48 0     64 @     80 P     96 `    112 p
   1 SOH     17 DC1     33 !        49 1     65 A     81 Q     97 a    113 q
   2 STX     18 DC2     34 "        50 2     66 B     82 R     98 b    114 r
   3 ETX     19 DC3     35 #        51 3     67 C     83 S     99 c    115 s
   4 EOT     20 DC4     36 $        52 4     68 D     84 T    100 d    116 t
   5 ENQ     21 NAK     37 %        53 5     69 E     85 U    101 e    117 u
   6 ACK     22 SYN     38 &        54 6     70 F     86 V    102 f    118 v
   7 BEL     23 ETB     39 '        55 7     71 G     87 W    103 g    119 w
   8 BS      24 CAN     40 (        56 8     72 H     88 X    104 h    120 x
   9 HT      25 EM      41 )        57 9     73 I     89 Y    105 i    121 y
  10 LF      26 SUB     42 *        58 :     74 J     90 Z    106 j    122 z
  11 VT      27 ESC     43 +        59 ;     75 K     91 [    107 k    123 {
  12 FF      28 FS      44 ,        60 <     76 L     92 \    108 l    124 |
  13 CR      29 GS      45 -        61 =     77 M     93 ]    109 m    125 }
  14 SO      30 RS      46 .        62 >     78 N     94 ^    110 n    126 ~
  15 SI      31 US      47 /        63 ?     79 O     95 _    111 o    127 DEL
```

“A” comes before ”Z” which comes before “a”
(65) (90) (97)

ORD

- The python built-in function ord(character) can be used to retrieve the ordinal number for a specified character
- Example: ord(“A”) ←returns 65

SELF CHECK

- What is the output of the following comparisons?
- `"BBB" < "aaa"`
- `"a" == "A"`
- `"9" > "8"`

SELF CHECK

- What is the output of the following comparisons?
- `"BBB" < "aaa"` **True**
- `"a" == "A"` **False**
- `"9" > "8"` **True**

TRY AT HOME

- Write a program to ask for the user's age. For those less than 18 output "Minor" otherwise output "Age of Majority"
- Program to read an integer from the user and print whether it is zero, negative or positive
- Program to compute and compare Area of Rectangles
  - Write code to determine if 2 rectangles have the same area:
  - Rectangle 1 dimensions: 2.49 x 2.49
  - Rectangle 2 dimensions: 1.0 x 6.2001
- There is a small inconsistency in a number comparison program. What is the inconsistency, given an example input that exposes it and then fix it.